

SECURITY CENTER AI

Guía de trabajo

Persona 2 — Frontend

Pasos para colaborar en GitHub, construir la interfaz en Next.js e integrar el login y el dashboard con la API.

Repositorio: github.com/omi12ff/SecurityCenterAI

Responsabilidad: Frontend, UI/UX, componentes, integración API y responsive.

Rama estable: `main`

Rama de integración: `develop`

1. Antes de comenzar

1. Enviar al propietario del repositorio tu nombre de usuario de GitHub.
2. Aceptar la invitación como colaborador desde GitHub o desde el correo.
3. Instalar Git, Node.js LTS y Visual Studio Code.
4. Configurar tu identidad de Git una sola vez:

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu-correo@example.com"
```

2. Descargar el proyecto

```
git clone https://github.com/omi12ff/SecurityCenterAI.git
cd SecurityCenterAI
git switch develop
git pull origin develop
```

Verifica que estés en la rama correcta:

```
git status
```

No programes directamente sobre `main` ni sobre `develop`. Cada tarea debe tener su propia rama `feature/*`.

3. Crear la rama para el frontend

```
git switch develop
git pull origin develop
git switch -c feature/frontend-base
```

Si otra persona ya creó esa rama, utiliza:

```
git fetch origin
git switch feature/frontend-base
```

4. Crear la aplicación Next.js

Desde la carpeta raíz `SecurityCenterAI`:

```
npx create-next-app@latest frontend --typescript --tailwind --eslint --app --src-dir --import-alias "@/*"
cd frontend
npm run dev
```

Abrir `http://localhost:3000`. Si aparece la pantalla inicial de Next.js, la instalación funciona.

Dependencias de interfaz recomendadas

```
npx shadcn@latest init
npx shadcn@latest add button card input label form alert badge
npm install react-hook-form zod @hookform/resolvers
```

5. Estructura sugerida

```
frontend/src/
├── app/
│   ├── (auth)/
│   │   ├── login/page.tsx
│   │   └── register/page.tsx
│   ├── dashboard/page.tsx
│   └── layout.tsx
├── components/
│   ├── auth/
│   ├── dashboard/
│   └── ui/
├── lib/
│   ├── api.ts
│   └── auth.ts
└── types/
    └── api.ts
```

6. Configurar la conexión con el backend

Crear `frontend/.env.local`:

```
NEXT_PUBLIC_API_URL=http://localhost:5000
```

Confirma el puerto real mostrado al ejecutar la API. No subas `.env.local` a GitHub. Next.js lo excluye normalmente, pero compruébalo con `git status`.

Cliente inicial de API

Crear `frontend/src/lib/api.ts`:

```
const API_URL = process.env.NEXT_PUBLIC_API_URL;

export async function apiFetch<T>({
  path: string,
  options: RequestInit = {}
}): Promise<T> {
  const response = await fetch(`${API_URL}${path}`, {
    ...options,
    headers: {
      "Content-Type": "application/json",
      ...options.headers,
    },
  });

  if (!response.ok) {
    const error = await response.json().catch(() => null);
    throw new Error(error?.title ?? "No se pudo completar la solicitud");
  }

  return response.json();
}
```

7. Contrato actual de la API

Función	Método y ruta	Autenticación
Registro	POST /api/auth/register	No
Login	POST /api/auth/login	No
Dashboard	GET /api/dashboard	Bearer JWT
Estado de API	GET /health	No

Registro

```
{
  "name": "Nombre Apellido",
  "email": "usuario@example.com",
  "password": "ClaveSegura123!"
}
```

Login

```
{
  "email": "usuario@example.com",
  "password": "ClaveSegura123!"
}
```

Respuesta de autenticación

```
{
  "accessToken": "jwt...",
  "expiresAtUtc": "2026-07-25T18:00:00Z",
  "user": {
    "id": "uuid",
    "name": "Nombre Apellido",
    "email": "usuario@example.com"
  }
}
```

Solicitud autenticada

```
const token = sessionStorage.getItem("accessToken");

const dashboard = await apiFetch("/api/dashboard", {
  headers: { Authorization: `Bearer ${token}` },
});
```

Para el MVP puede utilizarse `sessionStorage`. Antes de producción se debe evaluar autenticación con cookies `HttpOnly` para reducir la exposición del token.

8. Primera entrega del frontend

Construir en este orden:

1. Layout general, colores, tipografía y navegación.
2. Página de registro con validación y mensajes de error.
3. Página de login y almacenamiento temporal del token.
4. Redirección al dashboard después del login.
5. Dashboard protegido con puntaje, total de análisis e historial reciente.
6. Estados de carga, error y dashboard sin análisis.
7. Diseño responsive para teléfono, tableta y escritorio.

Criterios de aceptación

- ☐ Registro válido redirige al dashboard.
- ☐ Correo duplicado muestra un mensaje comprensible.
- ☐ Credenciales incorrectas no permiten entrar.
- ☐ Dashboard envía el JWT en la cabecera Authorization.
- ☐ La interfaz funciona desde 360 px de ancho.
- ☐ No hay errores de TypeScript, ESLint ni consola.

9. Commits y publicación de la rama

Revisar exactamente qué se modificó:

```
git status
git diff
```

Crear commits pequeños:

```
git add frontend
git commit -m "Add frontend base with Next.js"
git push -u origin feature/frontend-base
```

Ejemplos para siguientes commits:

```
git commit -m "Add login and registration forms"
git commit -m "Integrate authentication API"
git commit -m "Add responsive security dashboard"
```

10. Pruebas antes del Pull Request

```
cd frontend
npm run lint
npm run build
```

También probar manualmente:

- Registro, login y logout.
- Errores del servidor y API apagada.
- Token inválido o vencido.
- Vista móvil y escritorio.
- Navegación por teclado y etiquetas de formularios.

11. Pull Request hacia develop

1. Abrir el repositorio en GitHub.
2. Seleccionar **Pull requests** → **New pull request**.
3. Elegir base `develop` y compare `feature/frontend-base`.
4. Explicar qué cambió, cómo probarlo y adjuntar capturas.
5. Solicitar revisión al compañero.
6. No fusionar hasta que las pruebas pasen y la revisión sea aprobada.

Plantilla de descripción:

```
## Cambios
- ...

## Cómo probar
1. ...

## Evidencia
- npm run lint: correcto
- npm run build: correcto
- Capturas: ...

## Pendientes
- ...
```

12. Cómo mantenerse actualizado

Antes de comenzar cada jornada:

```
git switch develop
git pull origin develop
git switch feature/frontend-base
git merge develop
```

Si aparecen conflictos, resolverlos con cuidado, probar nuevamente y crear un commit. Nunca utilizar comandos destructivos para “borrar” conflictos.

13. Reglas del equipo

- `main` contiene únicamente versiones estables.
- `develop` reúne funcionalidades revisadas para pruebas.
- Cada funcionalidad nace desde `develop` en una rama `feature/*`.
- Nadie aprueba su propio Pull Request.
- Los secretos nunca se guardan en Git.
- Un cambio de contrato de API se comunica antes de implementarlo.
- Swagger es la referencia del contrato backend.
- Los errores se describen con pasos para reproducirlos.

14. Definición de “terminado”

Una tarea está terminada cuando funciona localmente, es responsive, pasa lint y build, maneja errores, está documentada, fue subida a una rama `feature/*` y recibió revisión mediante Pull Request.

Resumen rápido

```
git switch develop
git pull origin develop
git switch -c feature/mi-tarea

# trabajar y probar

git status
git add frontend
git commit -m "Descripción breve"
git push -u origin feature/mi-tarea

# abrir PR: feature/mi-tarea → develop
```